

Manual de Arquitetura e Implementação Avançada da API BaseLinker

1. Visão Geral e Filosofia Arquitetural

A plataforma BaseLinker opera como um orquestrador central (middleware) para operações de comércio eletrônico, unificando a comunicação entre marketplaces (como Amazon, eBay, Allegro), lojas online (Shopify, Magento, WooCommerce), transportadoras e sistemas de gestão empresarial (ERP). A Interface de Programação de Aplicações (API) do BaseLinker é o mecanismo fundamental que permite esta interoperabilidade, oferecendo um canal bidirecional para a sincronização de dados críticos de negócio.

Diferentemente das arquiteturas RESTful (Representational State Transfer) puristas, que segregam operações através de verbos HTTP distintos (GET para leitura, POST para criação, PUT para atualização, DELETE para remoção) e utilizam URLs semânticas para recursos, a API do BaseLinker adota um paradigma arquitetural que se assemelha mais a um RPC (Remote Procedure Call) sobre HTTP. Todas as interações com o sistema são canalizadas através de um único ponto de entrada (endpoint), independentemente da natureza da operação — seja ela a recuperação de uma lista de encomendas ou a criação de uma etiqueta de envio.¹

Esta abordagem de design simplifica a configuração de rotas no lado do cliente, pois não exige a gestão de múltiplos endpoints, mas transfere a responsabilidade da definição da ação para o corpo da requisição (payload). A comunicação é estritamente realizada através do método HTTP POST, utilizando o formato JSON (JavaScript Object Notation) para a estruturação de dados, codificados em UTF-8. A robustez desta integração depende intrinsecamente do cumprimento rigoroso de três pilares técnicos: a autenticação baseada em tokens, a estruturação precisa do payload JSON e a gestão proativa dos limites de taxa (rate limiting) impostos pela infraestrutura.¹

1.1. O Endpoint Único e Protocolo de Comunicação

Toda a comunicação com a API do BaseLinker converge para um único URL. Esta centralização exige que o integrador especifique a "intenção" da chamada através de parâmetros no corpo da requisição.

Endpoint de Produção:

<https://api.baselinker.com/connector.php>

A natureza síncrona da maioria das chamadas exige que o sistema cliente aguarde a confirmação de processamento (ou erro) antes de prosseguir, embora certas operações em massa possam ter tempos de execução mais longos. O protocolo exige que os dados sejam enviados via POST, e embora o formato de troca de dados seja JSON, a camada de transporte espera frequentemente que os parâmetros sejam encapsulados num formato

application/x-www-form-urlencoded ou multipart/form-data, onde o JSON é o valor de um campo específico denominado parameters.¹

1.2. Codificação e Tratamento de Caracteres

A integridade dos dados é mantida através da codificação UTF-8. No entanto, uma nuance crítica na implementação refere-se ao tratamento de dados binários ou strings especiais. Para endpoints que aceitam ficheiros (como o upload de faturas externas ou a recuperação de etiquetas em PDF), a API utiliza codificação Base64. É imperativo que, ao transmitir estes dados, o caractere "+" (comum em Base64) seja substituído pela sequência de escape "%2B" antes da submissão. A falha nesta substituição resulta frequentemente em erros de decodificação no servidor, corrompendo o ficheiro transmitido.¹

2. Protocolos de Segurança e Autenticação

A segurança na API BaseLinker é gerida através de um modelo de token de acesso estático. Este modelo "stateless" (sem estado) alinha-se com as práticas modernas de microserviços, onde cada requisição deve ser auto-contida e autenticada independentemente, sem depender de sessões de servidor ou cookies.³

2.1. Gestão e Ciclo de Vida do Token

O token da API não é gerado programaticamente através de um fluxo de login (como OAuth2), mas sim atribuído diretamente à conta do utilizador no painel administrativo do BaseLinker.

- **Geração:** O token é obtido na secção "Conta e outros -> A minha conta -> API".
- **Escopo de Acesso:** É crucial compreender que este token concede permissões administrativas completas sobre a conta. Ele permite ler dados sensíveis de clientes (PII - Personally Identifiable Information), alterar stocks e criar envios. Portanto, deve ser tratado com o nível de segurança de uma "chave mestra". O comprometimento deste token exige a sua revogação imediata e a regeneração de um novo, o que implicará a atualização de todas as integrações dependentes.¹

2.2. Métodos de Transmissão do Token

Historicamente, a API permitia o envio do token como um parâmetro dentro do corpo da requisição POST (campo token). No entanto, esta prática é atualmente classificada como **depreciada (deprecated)** e insegura. O envio de credenciais no corpo da requisição expõe o segredo em logs de servidores proxy, balanceadores de carga e ferramentas de monitorização que frequentemente registam o payload completo das mensagens HTTP.

Método Recomendado (Header HTTP):

A implementação correta e segura exige o envio do token através de um cabeçalho HTTP personalizado.

Cabeçalho	Valor	Descrição
X-BLToken	''	Cabeçalho proprietário para autenticação.

Este método garante que o token seja encriptado durante o transporte (via HTTPS) e, geralmente, não seja registado nos logs de acesso padrão dos servidores web, aumentando significativamente a postura de segurança da integração.¹

3. Gestão de Tráfego e Rate Limiting (Limites de Taxa)

Para garantir a estabilidade e a disponibilidade da plataforma para todos os utilizadores, o BaseLinker impõe limites estritos ao volume de requisições que podem ser processadas por unidade de tempo. A violação destes limites resulta em bloqueios temporários, o que pode interromper fluxos críticos de negócio, como a atualização de stocks ou o download de encomendas.

3.1. Definição do Limite

O limite oficial é de **100 requisições por minuto** por conta de utilizador. É fundamental notar que este limite é agregado ao nível da conta, não do endereço IP ou da integração específica. Se um utilizador tiver três integrações distintas (ex: um ERP, uma ferramenta de análise de dados e um sincronizador de stock) a operar simultaneamente, todas partilham o mesmo "orçamento" de 100 requisições por minuto.¹

3.2. Estratégias de Mitigação e Algoritmos de Cliente

Para operar de forma fiável dentro destes limites, os arquitetos de software devem implementar estratégias de controlo de fluxo no lado do cliente.

Algoritmo Token Bucket

A implementação de um algoritmo "Token Bucket" ou "Leaky Bucket" é altamente recomendada. Neste modelo, o cliente mantém um contador local de tokens disponíveis. Cada requisição consome um token. O "balde" é reabastecido a uma taxa constante (neste caso, 1.66 tokens por segundo). Se o balde estiver vazio, a requisição deve ser colocada em fila de espera (queued) no lado do cliente, em vez de ser enviada para a API.⁵

Backoff Exponencial

Em caso de recebimento de um erro HTTP 429 (Too Many Requests), a aplicação não deve tentar reenviar a requisição imediatamente. Deve-se adotar uma estratégia de "Backoff Exponencial", onde o tempo de espera entre as tentativas duplica progressivamente (ex: 1s,

2s, 4s, 8s). Isto permite que a janela de tempo de bloqueio expire e alivia a carga sobre o servidor.⁷

Otimização via Bulk Operations

A forma mais eficiente de contornar os limites de taxa não é aumentar a frequência das chamadas, mas sim aumentar a densidade de dados por chamada. A API do BaseLinker suporta operações em lote para funções críticas. Por exemplo, a atualização de preços (updateInventoryProductsPrices) permite modificar até 1000 produtos numa única transação. Utilizar este método consome apenas 1 crédito do limite de taxa, enquanto atualizar os mesmos produtos individualmente consumiria 1000 créditos e levaria 10 minutos para ser concluído sob as restrições atuais.¹

4. Módulo de Gestão de Encomendas (Order Management)

O módulo de encomendas é o centro nevrálgico de qualquer operação de e-commerce. A API permite a extração de encomendas de múltiplas fontes, a sua injeção no sistema e a manipulação dos seus estados, servindo como ponte vital entre as vendas front-end e a logística back-end.

4.1. Recuperação e Sincronização de Encomendas (getOrders)

A função getOrders é a ferramenta primária para extrair dados de vendas. A sua correta utilização é crítica para evitar a perda de encomendas ou a duplicação de processamento.

Estrutura da Requisição:

Parâmetro	Tipo	Obrigatoriedade	Descrição Técnica e Impacto
order_id	Int	Opcional	Filtra por um ID específico. Útil para "refresh" pontual de dados.
date_confirmed_from	Int (Unix)	Crítico	Define o marco temporal inicial (com base na data de confirmação) para a

			recuperação.
date_from	Int (Unix)	Opcional	Filtra pela data de criação (date_add). Menos fiável para sync, pois encomendas podem ser criadas mas confirmadas muito depois.
get_unconfirmed_orders	Bool	Opcional (Default: False)	Se true, baixa encomendas incompletas. Recomendação: Manter false para garantir a integridade dos dados no ERP.
status_id	Int	Opcional	Filtra encomendas num estado específico (ex: apenas "Pagas").

O Problema da Paginação e a Solução de Travessia Temporal:

Em APIs tradicionais, a paginação é feita via offset (página 1, página 2). No entanto, em sistemas de e-commerce dinâmicos, o uso de páginas é arriscado: se novas encomendas entrarem enquanto o script está a ler a página 1, o offset muda e encomendas podem ser "empurradas" para páginas já lidas, resultando em dados perdidos.

A documentação do BaseLinker preconiza uma abordagem de sincronização contínua baseada em timestamps (date_confirmed), que elimina este risco:

1. O sistema cliente armazena localmente o timestamp da última encomenda processada com sucesso.
2. Na requisição seguinte, envia este valor no campo date_confirmed_from.
3. A API retorna as encomendas confirmadas após esse segundo.
4. Se a resposta contiver 100 encomendas (o limite máximo por resposta), o cliente deve imediatamente solicitar o próximo lote, usando o timestamp da *última* encomenda recebida + 1 segundo como novo ponto de partida.
5. O ciclo repete-se até que a API retorne menos de 100 resultados, indicando que o buffer

está vazio.¹

Análise dos Dados de Resposta:

A resposta fornece um objeto complexo para cada encomenda.

- **Identificadores:** É vital distinguir entre `order_id` (o ID único gerado pelo BaseLinker), `shop_order_id` (o ID na plataforma de origem, como o #1001 do Magento) e `external_order_id` (o ID da transação no marketplace, ex: número da transação eBay ou ID do pedido Allegro). O `order_id` deve ser usado para todas as operações subsequentes na API.
- **Dados Financeiros:** A API retorna `payment_done` (valor pago) e `want_invoice` (indicador de necessidade de fatura). O campo `currency` indica a moeda da transação, o que é crucial para lojas multi-moeda.¹¹

4.2. Injeção e Criação de Encomendas (addOrder)

A função `addOrder` permite a criação manual de encomendas no sistema. Isto é frequentemente utilizado para integrar vendas realizadas em canais offline (lojas físicas, telefone) ou importadas de ficheiros CSV legados.

Ao criar uma encomenda, certos campos são mandatórios para garantir que esta possa ser processada logisticamente:

- `email`, `phone`, `delivery_fullname`, `delivery_address`, `delivery_city`, `delivery_postcode`: Dados essenciais para a geração de etiquetas de transporte.
- `products`: Um array contendo os itens da encomenda. Cada item deve referenciar um `product_id` (se existir no catálogo) ou fornecer `name`, `price_brutto` e `quantity` para itens ad-hoc.¹²

O retorno desta função é o `order_id` da nova encomenda, que deve ser imediatamente registado pelo sistema emissor para referência futura.¹²

4.3. Manipulação de Estado e Workflow (setOrderStatus)

O BaseLinker opera com base em estados de encomenda (ex: "Novo", "Em Processamento", "Enviado", "Concluído"). A API permite alterar estes estados programaticamente através de `setOrderStatus`.

Cenário de Automação:

Uma utilização comum envolve a integração com um WMS (Warehouse Management System). Quando o operador do armazém digitaliza o código de barras da encomenda e a fecha para envio, o WMS envia uma chamada `setOrderStatus` para o BaseLinker, movendo a encomenda para o estado "Pronto para Envio". Isto, por sua vez, pode disparar "Ações Automáticas" no painel do BaseLinker, como o envio de um e-mail ao cliente ou a impressão da fatura.¹

4.4. Operações Avançadas em Encomendas

Para cenários complexos, a API oferece métodos granulares:

- **addOrderBySplit:** Utilizado para gerir *backorders*. Se uma encomenda tem 5 itens e apenas 3 estão em stock, este método permite mover os 2 itens em falta para uma nova encomenda, permitindo o envio parcial imediato.¹⁵
- **addOrderProduct / deleteOrderProduct:** Permitem a edição do conteúdo de uma encomenda já existente. Crítico para equipas de suporte ao cliente que precisam alterar cores ou tamanhos a pedido do cliente antes do envio.¹

5. Arquitetura de Inventário e Catálogo de Produtos

O BaseLinker adota uma arquitetura híbrida para a gestão de produtos, distinguindo claramente entre o seu próprio catálogo interno (BaseLinker Inventory) e os catálogos de sistemas externos (Lojas, Atacadistas, ERPs). Compreender esta distinção é vital para evitar erros de sincronização de stock.

5.1. O Catálogo Nativo (Inventory)

Quando o BaseLinker é utilizado como PIM (Product Information Manager) central, os produtos residem na sua base de dados interna. As funções da API para este contexto contêm o prefixo Inventory.

Gestão de Dados do Produto

- **addInventoryProduct:** Este método é idempotente em relação à atualização; ele cria um novo produto ou atualiza um existente se o ID for fornecido. Suporta a definição complexa de produtos, incluindo:
 - **Múltiplos Stocks:** Definição de quantidades em diferentes armazéns físicos via array stock.
 - **Preços Multi-Canal:** Definição de preços para diferentes grupos de clientes ou canais de venda via array prices.
 - **Variantes:** Suporte para produtos "Pai" e "Filhos" (variantes de cor/tamanho), geridos através do parent_id.¹⁷
- **getInventoryProductsData:** Método robusto para extração de dados. Permite recuperar não apenas os dados básicos (sku, ean), mas também estruturas aninhadas como text_fields (descrições traduzidas), locations (localização no armazém) e links (associações com lojas externas).¹⁸

Sincronização de Alta Performance

Para integradores de ERPs que necessitam de refletir alterações de stock e preço em tempo real, o uso de métodos individuais (addInventoryProduct) é ineficiente e propenso a atingir os limites de taxa. O BaseLinker oferece métodos otimizados para operações em massa:

- **updateInventoryProductsStock:** Permite atualizar o stock de até **1000 produtos** numa

única requisição HTTP. O payload é um mapa simplificado de `product_id/variant_id` e a nova quantidade.

- **updateInventoryProductsPrices:** Análogo à atualização de stock, permite alterar preços de 1000 SKUs simultaneamente.

Implicação Arquitetural: Ao desenhar um sincronizador, deve-se acumular alterações no lado do ERP num buffer (por exemplo, durante 1 ou 2 minutos) e enviá-las num único lote utilizando estes métodos, em vez de disparar uma requisição API a cada venda individual.¹

5.2. Armazéns Externos (ExternalStorage)

Muitas empresas utilizam o BaseLinker apenas como um "pass-through", mantendo a "fonte da verdade" dos produtos numa loja Magento, PrestaShop ou num atacadista conectado. Neste cenário, os produtos não residem na base de dados do BaseLinker, mas são acedidos em tempo real através dele.

- **getProductsData:** Recupera dados diretamente da API da loja conectada. Exige o parâmetro `storage_id` (ex: `shop_2445`) para encaminhar a consulta. É útil para obter dados brutos como IDs de categorias da loja ou descrições em HTML específicas da plataforma.¹⁹
- **updateExternalStorageProductsQuantity:** Permite que o BaseLinker atue como agente de escrita, atualizando o stock na loja remota. Isto é comum quando o BaseLinker gere o stock centralizado e precisa propagar a redução de inventário para as lojas satélites.¹

6. Módulo de Logística e Integração com Transportadoras (Couriers)

O módulo de correios da API é uma abstração poderosa (Padrão de Projeto *Facade*) que normaliza a comunicação com dezenas de transportadoras diferentes (DHL, DPD, UPS, FedEx, CTT), apresentando uma interface unificada para o desenvolvedor.

6.1. O Fluxo de Criação de Envios

Devido à diversidade de requisitos operacionais entre transportadoras (algumas exigem dimensões exatas, outras apenas peso; algumas exigem pontos de recolha, outras morada de porta), a criação de uma etiqueta de envio não é uma chamada atómica, mas um processo de descoberta e submissão.

Fase 1: Descoberta de Metadados

Antes de criar o pacote, a aplicação deve consultar quais os parâmetros exigidos pela transportadora selecionada.

- **getCouriersList:** Retorna os códigos das transportadoras ativas na conta (ex: `dhl`, `ups`).
- **getCourierFields:** **Método Crítico.** Retorna a definição do formulário para uma

transportadora específica. A resposta inclui uma lista de campos (fields) que podem ser caixas de texto, seletores (dropdowns) ou checkboxes. Por exemplo, para a DPD, este método pode indicar que o campo `service_type` é obrigatório e fornecer os valores válidos (ex: "Next Day", "Classic").¹

Fase 2: Criação do Pacote (`createPackage`)

Com os metadados em mãos, a aplicação constrói o payload.

- **Payload:** Deve incluir o `order_id` e um array `fields` preenchido com os valores validados na fase anterior. Também inclui o array `packages` com pesos e dimensões físicas.
- **Resposta:** Em caso de sucesso, retorna o `package_id` (interno) e o `package_number` (código de rastreio/tracking number).²⁰

Fase 3: Obtenção de Documentos

- **getLabel:** Utiliza o `package_id` para obter a etiqueta de envio. O formato (PDF, ZPL, EPL) depende da configuração da transportadora.
- **getProtocol:** Gera o manifesto de carga ou protocolo de entrega, documento essencial para a recolha física dos pacotes pelo estafeta.¹

6.2. Envios Manuais e Externos (`createPackageManual`)

Para operações onde a etiqueta é gerada fora do ecossistema BaseLinker (ex: portal da transportadora ou logística própria), utiliza-se `createPackageManual`. Este método associa manualmente um número de rastreio e nome de transportadora a uma encomenda, permitindo que o BaseLinker envie os e-mails transacionais de "Enviado" com o link de rastreio correto para o cliente final.²¹

7. Módulo Financeiro e Faturação

A API permite integrar o ciclo de faturação, seja utilizando o motor interno do BaseLinker ou conectando-se a softwares de faturação externos (SAFT).

7.1. Emissão de Documentos Fiscais

- **addInvoice:** Dispara a emissão de uma fatura para uma encomenda específica. Requer a seleção de uma série de numeração (`series_id`) e o método de cálculo do IVA (`vat_rate`). O sistema pode herdar as taxas de IVA dos produtos ou forçar isenções (ex: para exportação).²²
- **addInvoiceCorrection:** Permite a emissão de notas de crédito ou débito. Pode ser baseada numa fatura existente (`original_invoice_id`) ou numa devolução de encomenda (`return_order_id`), garantindo a rastreabilidade fiscal completa.²³

7.2. Integração de Faturas Externas (`addOrderInvoiceFile`)

Para empresas que utilizam ERPs robustos (SAP, PHC, Primavera) para a emissão fiscal, o BaseLinker não deve emitir a fatura, mas deve disponibilizá-la ao cliente. O método `addOrderInvoiceFile` permite fazer upload de um ficheiro PDF (gerado externamente) e associá-lo à encomenda no BaseLinker. Desta forma, o marketplace ou a loja online podem apresentar a fatura oficial ao cliente através da API do BaseLinker, mantendo a consistência da experiência do utilizador.¹

8. Tratamento de Erros e Diagnóstico

A robustez de uma integração define-se pela sua capacidade de lidar com falhas. A API do BaseLinker apresenta comportamentos específicos de erro que devem ser tratados programaticamente.

8.1. Taxonomia de Erros

Existem duas camadas de erro na comunicação com a API:

1. Erros de Transporte (HTTP Level):

Indicam falhas na infraestrutura ou violação de protocolos de rede.

- **400 Bad Request:** Sintaxe JSON inválida ou cabeçalhos mal formados.
- **429 Too Many Requests:** Violação do limite de 100 req/min. Exige implementação de espera e re-tentativa (retry).
- **500 Internal Server Error:** Falha interna do servidor ou timeout de processamento (frequentemente causado por operações em lote excessivamente grandes).
- **401 Unauthorized / 403 Forbidden:** Problemas com o Token de API.⁷

2. Erros de Aplicação (JSON Level):

Muitas vezes, a API retorna um código HTTP 200 OK, mas a operação falhou logicamente. O corpo da resposta conterá um campo `status` com o valor `ERROR`.

Código de Erro (Exemplo)	Descrição e Ação Recomendada
ERROR_BAD_TOKEN	O token fornecido não existe ou é inválido. Ação: Verificar configuração e regenerar token.
ERROR_METHOD_NOT_FOUND	O nome do método no payload está incorreto. Ação: Verificar a grafia exata na documentação.
ERROR_ORDER_NOT_FOUND	Tentativa de atualizar uma encomenda que não existe. Ação: Verificar se o ID é o

	order_id interno e não o externo.
ERROR_STORAGE_ID	O ID do armazém/loja fornecido é inválido. Ação: Consultar getStoragesList.

8.2. Boas Práticas de Logging

É imperativo que qualquer integração registre não apenas o código HTTP, mas também o request_id (se fornecido nos headers) e o corpo completo da resposta JSON em caso de erro. Dado que o suporte técnico do BaseLinker necessita frequentemente do timestamp exato e dos parâmetros enviados para diagnosticar problemas, logs detalhados são a principal ferramenta de resolução de problemas.²⁴

9. Padrões de Integração e Melhores Práticas

A construção de uma integração resiliente exige mais do que o conhecimento dos métodos individuais; exige a aplicação de padrões arquiteturais corretos.

9.1. Polling vs. Eventos

A API do BaseLinker não suporta Webhooks nativos para todas as alterações de dados (embora suporte "Ações Automáticas" que podem chamar URLs externos). Portanto, a sincronização de dados depende frequentemente de **Polling Inteligente**.

- **Monitorização de Encomendas:** Utilize o método de travessia temporal (getOrders com date_confirmed) a cada 5 ou 10 minutos.
- **Monitorização de Eventos de Log:** Para sistemas que precisam de auditar quem fez o quê, o método getJournalList (referenciado na documentação, embora exija ativação via suporte em alguns casos) permite descarregar logs de operações, como "Usuário X alterou o stock do produto Y".¹

9.2. Identificação de Origens (getOrderSources)

Em ambientes omnicanal, é vital saber de onde veio uma encomenda para aplicar regras de negócio específicas (ex: "Se vier da Amazon, usar fatura sequencial A; se vier do Site, usar sequencial B"). O método getOrderSources fornece o mapeamento entre os IDs numéricos de fonte e os nomes legíveis (Amazon, eBay), permitindo esta lógica condicional no código de integração.¹

9.3. Bibliotecas de Cliente (SDKs)

Embora a integração direta via cURL seja possível, o uso de bibliotecas de comunidade pode

acelerar o desenvolvimento ao abstrair a autenticação e serialização JSON.

- **PHP:** Bibliotecas como baselinker-php oferecem interfaces fluentes (`$baselinker->orders()->getOrders(...)`) e gestão automática de exceções.²⁵
- **Python:** Módulos como py-baselinker são ideais para scripts de automação e análise de dados.²⁷
- **Go:** go-baselinker é recomendado para microsserviços de alta performance.²⁹
Nota: Recomenda-se sempre auditar o código destas bibliotecas antes da utilização em produção para garantir a segurança do token.

10. Conclusão

A API do BaseLinker apresenta-se como uma solução robusta e abrangente para a complexidade do comércio unificado. A sua arquitetura centrada em métodos RPC sobre JSON permite um controlo granular sobre todos os aspetos da operação, desde o catálogo até à expedição. No entanto, o seu poder vem acompanhado da responsabilidade de uma implementação cuidadosa, especialmente no que tange à segurança do token, ao respeito pelos limites de taxa e à lógica de sincronização de dados.

Para o arquiteto de software, o sucesso na integração com o BaseLinker reside não apenas na codificação das chamadas, mas no desenho de processos que tolerem a assincronia, validem a integridade dos dados e recuperem graciosamente de falhas. Seguindo as diretrizes e padrões detalhados neste relatório, é possível construir ecossistemas de e-commerce altamente escaláveis e automatizados, onde o BaseLinker atua eficazmente como o motor invisível que impulsiona as vendas globais.

Referências citadas

1. API documentation - baselinker.com, acessado em janeiro 13, 2026, <https://api.baselinker.com/>
2. baselinker.com - API documentation, acessado em janeiro 13, 2026, <https://api.baselinker.com/?tester&method=getConnectIntegrations>
3. A guide to REST API authentication - Merge.dev, acessado em janeiro 13, 2026, <https://www.merge.dev/blog/rest-api-authentication>
4. Rate limits and user limits - Hyperliquid Docs - GitBook, acessado em janeiro 13, 2026, <https://hyperliquid.gitbook.io/hyperliquid-docs/for-developers/api/rate-limits-and-user-limits>
5. What is Rate Limiting? - Cyclr, acessado em janeiro 13, 2026, <https://cyclr.com/resources/api-integration/what-is-rate-limiting>
6. REST Rate Limits Overview - Coinbase Developer Documentation, acessado em janeiro 13, 2026, <https://docs.cdp.coinbase.com/exchange/rest-api/rate-limits>
7. Understanding API Error Codes: 10 Status Errors When Building APIs For The First

Time And How To Fix Them - Moesif, acessado em janeiro 13, 2026,
<https://www.moesif.com/blog/technical/monitoring/Understanding-API-Error-Codes-10-Status-Errors-When-Building-APIs-For-The-First-Time-And-How-To-Fix-Them/>

8. Understanding Common API Error Codes: A 2024 Guide, acessado em janeiro 13, 2026,
<https://www.knowl.ai/blog/understanding-common-api-error-codes-a-2024-guide-clszofsizr003s12x1vxusu5re>
9. updateProductsPrices - API documentation - baselinker.com, acessado em janeiro 13, 2026,
<https://api.baselinker.com/index.php?method=updateProductsPrices>
10. Limits - Developer Platform Overview - Benchling, acessado em janeiro 13, 2026,
<https://docs.benchling.com/docs/rate-limiting>
11. getOrders - API documentation - baselinker.com, acessado em janeiro 13, 2026,
<https://api.baselinker.com/index.php?method=getOrders>
12. API documentation - baselinker.com, acessado em janeiro 13, 2026,
<https://api.base.com/index.php?method=addOrder>
13. addOrder - API documentation - baselinker.com, acessado em janeiro 13, 2026,
<https://api.baselinker.com/index.php?method=addOrder>
14. setOrderStatus - API documentation - baselinker.com, acessado em janeiro 13, 2026,
<https://api.baselinker.com/index.php?method=setOrderStatus>
15. baselinker.com - API documentation, acessado em janeiro 13, 2026,
<https://api.baselinker.com/?tester&method=addOrderBySplit>
16. addOrderProduct - API documentation - baselinker.com, acessado em janeiro 13, 2026,
<https://api.baselinker.com/index.php?method=addOrderProduct>
17. addInventoryProduct - API documentation - baselinker.com, acessado em janeiro 13, 2026,
<https://api.baselinker.com/index.php?method=addInventoryProduct>
18. getInventoryProductsData - API documentation - baselinker.com, acessado em janeiro 13, 2026,
<https://api.baselinker.com/index.php?method=getInventoryProductsData>
19. getProductsData - API documentation - baselinker.com, acessado em janeiro 13, 2026,
<https://api.baselinker.com/index.php?method=getProductsData>
20. createPackage - API documentation - baselinker.com, acessado em janeiro 13, 2026,
<https://api.baselinker.com/index.php?method=createPackage>
21. createPackageManual - API documentation - baselinker.com, acessado em janeiro 13, 2026,
<https://api.baselinker.com/index.php?method=createPackageManual>
22. addInvoice - API documentation - baselinker.com, acessado em janeiro 13, 2026,
<https://api.baselinker.com/index.php?method=addInvoice>
23. addInvoiceCorrection - API documentation - baselinker.com, acessado em janeiro 13, 2026,
<https://api.baselinker.com/index.php?method=addInvoiceCorrection>
24. 7 best practices for building and maintaining API integrations - Merge.dev, acessado em janeiro 13, 2026,
<https://www.merge.dev/blog/api-integration-best-practices>

25. Solizion/php-baselinker - GitHub, acessado em janeiro 13, 2026,
<https://github.com/Solizion/php-baselinker>
26. mnastalski/baselinker-php: PHP library for Base.com (formerly BaseLinker) API - GitHub, acessado em janeiro 13, 2026,
<https://github.com/mnastalski/baselinker-php>
27. Solizion/py-baselinker: Python library to communication with baselinker - GitHub, acessado em janeiro 13, 2026, <https://github.com/Solizion/py-baselinker>
28. michalkulisiewicz/python-baselinker - GitHub, acessado em janeiro 13, 2026,
<https://github.com/michalkulisiewicz/python-baselinker>
29. baselinker package - github.com/oxess/go-baselinker - Go Packages, acessado em janeiro 13, 2026, <https://pkg.go.dev/github.com/oxess/go-baselinker>